

NextStop

Government Bus Tracking & Fleet Management System

System Design Capstone Project Report

Name	Role
Sanchit Garg	Frontend Developer
Tanish Garg	Backend Developer
Omkar Shukla	Backend Developer
Chinmay Soni	QA / Documentation
Raghav Gupta	DevOps / Integration

May 2026

0 Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Objectives	2
2	Technology Stack	2
3	Software Development Life Cycle (SDLC)	3
3.1	Phase 1 – Planning (Week 1–2)	3
3.2	Phase 2 – Design (Week 2–3)	3
3.3	Phase 3 – Development (Week 3–8)	3
3.4	Phase 4 – Testing (Continuous)	4
3.5	Phase 5 – Deployment	4
4	System Architecture	4
4.1	Layered MVC + Service Layer	4
4.2	Real-Time Data Flow	4
4.3	Multi-Tier Authentication	5
5	UML and Data Modelling Diagrams	5
5.1	Use Case Diagram	5
5.2	Class Diagram	5
5.3	Entity-Relationship (ER) Diagram	5
6	Object-Oriented Programming Concepts	6
6.1	Encapsulation	6
6.2	Inheritance	6
6.3	Polymorphism	7
6.4	Abstraction	7
7	Design Patterns (Gang of Four)	8
7.1	Singleton Pattern	8
7.2	Observer Pattern	8
7.3	Facade Pattern	9
7.4	Strategy Pattern	9
8	API Endpoints	10
8.1	ETM Device (/etm/v1) – X-Device-Key required	10
8.2	Passenger App (/app/v1) – App Token required	10
8.3	Conductor (/conductor/v1) – Conductor Token required	10
8.4	Admin (/admin/v1) – Admin Token required	11
9	Key Features Delivered	11
10	Conclusion	12

1 Introduction

NextStop is a full-stack, real-time public transport management platform built for government bus agencies. It provides live GPS tracking, fleet and route management, ticketing workflows, and operational analytics through a unified web interface.

The system supports three distinct user roles:

- **Passengers** – Real-time bus tracking, route search, QR-based ticket booking.
- **Conductors** – On-bus ticket issuance, trip management, offline data sync.
- **Administrators** – Fleet oversight, route CRUD, analytics dashboards.

1.1 Problem Statement

Traditional government bus systems suffer from:

1. No real-time visibility into bus locations or active trips.
2. Manual, error-prone route and fleet management.
3. Limited analytics for revenue planning and demand forecasting.
4. Poor coordination between buses, drivers, and conductors.

NextStop addresses all four gaps by combining IoT telemetry ingestion, centralised administration, and actionable analytics in a single platform.

1.2 Objectives

- Build a real-time bus tracking system using GPS and SSE streaming.
- Manage routes, buses, drivers, and conductors from a single admin portal.
- Support ticketing and trip operations through dedicated backend services.
- Provide revenue, route-performance, and demand analytics.
- Demonstrate clean architecture using OOP principles and GoF design patterns.

2 Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 16 (React 19)	Admin Portal SPA
Styling	Tailwind CSS 4, Radix UI	UI components & icons
Charts	Recharts, Chart.js	Data visualisation
Maps	OLA Maps Web SDK	Geospatial live tracking
Backend	Node.js + Express.js 4	REST API server
Language	TypeScript 5.3	Type-safe backend
Database	MongoDB 7+ (Mongoose 8)	NoSQL document store
Validation	Zod 3	Runtime schema validation
Real-time	SSE (Server-Sent Events)	1-way push from server
Media	Cloudinary	Image / audio storage

Layer	Technology	Purpose
QR Codes	qrcode (Node.js)	Ticket QR generation
Cloud	MongoDB Atlas, Vercel, Railway	Hosting & database

3 Software Development Life Cycle (SDLC)

The project follows an **Agile Iterative** model with five major phases executed over eight weeks.

3.1 Phase 1 – Planning (Week 1–2)

Requirements were gathered and classified by priority:

ID	Requirement	Priority
FR-1	Real-time GPS tracking of buses	High
FR-2	Conductor ticket issuance	High
FR-3	Passenger booking with QR codes	High
FR-4	Admin dashboard with fleet management	High
FR-5	Route management (CRUD + bulk upload)	Medium
FR-6	Revenue and ticket analytics	Medium
FR-7	Offline ticket sync	Medium
FR-8	Voice-based route queries (Punjabi)	Low

3.2 Phase 2 – Design (Week 2–3)

Key architecture decisions:

Decision	Choice	Rationale
Frontend Framework	Next.js 16	SSR, App Router, excellent DX
Backend Framework	Express.js	Lightweight, flexible ecosystem
Database	MongoDB	Schema flexibility, geospatial support
Real-time Comm.	SSE	Simpler than WebSockets for 1-way push
Validation	Zod	Runtime type safety with TypeScript
Maps	OLA Maps SDK	India-focused local mapping

3.3 Phase 3 – Development (Week 3–8)

Five sprints were executed:

Sprint 1 Foundation – MongoDB models, Express server, Next.js setup, auth.

Sprint 2 Core features – ETM ingestion, admin dashboard, route and fleet management, OLA Maps.

Sprint 3 Multi-portal APIs – Passenger app, QR booking, conductor endpoints, offline sync.

Sprint 4 Analytics & SSE – Revenue charts, demand analysis, real-time SSE streaming.

Sprint 5 Polish – Responsive design, error handling, database seeder, documentation.

Metric	Count
Backend Models	14
API Route Files	8
Service Modules	10
Frontend Components	16+
Frontend Pages	6
API Endpoints	40+

3.4 Phase 4 – Testing (Continuous)

Testing strategies used:

1. **Manual API Testing** via Postman – all 40+ endpoints.
2. **Frontend Testing** – cross-browser and responsive layout checks.
3. **Integration Testing** – end-to-end user flows.
4. **Data Validation** – Zod schema validation on all inputs.

3.5 Phase 5 – Deployment

Component	Platform
Frontend	Vercel / VPS (Port 8080)
Backend	Railway / VPS (Port 3001)
Database	MongoDB Atlas
Media	Cloudinary

4 System Architecture

4.1 Layered MVC + Service Layer

The project follows a four-tier layered architecture:

1. **View (Client Layer)** – Next.js React components render the admin portal.
2. **Controller Layer** – Express.js route handlers validate HTTP requests and return responses.
3. **Service Layer** – Business logic is isolated in service classes, keeping controllers thin.
4. **Model Layer** – Mongoose schemas define 14 MongoDB collections.

4.2 Real-Time Data Flow

1. Bus ETM device sends `POST /etm/v1/heartbeat` with GPS coordinates every ~5 seconds.

2. `IngestService` validates and persists the `Heartbeat` document in MongoDB.
3. `EventBus.notify()` is called, broadcasting the update to all SSE subscribers.
4. Admin browser receives the push event and moves the bus marker on the OLA Map.

4.3 Multi-Tier Authentication

User Role	Auth Mechanism	Strategy Class
Admin	Bearer token (email/password login)	<code>AdminAuthStrategy</code>
Passenger	APP_ prefixed token	<code>AppUserAuthStrategy</code>
Conductor	CONDUCTOR_ prefixed token	<code>ConductorAuthStrategy</code>
IoT Device	X-Device-Key header	<code>DeviceAuthStrategy</code>

5 UML and Data Modelling Diagrams

5.1 Use Case Diagram

The Use Case Diagram captures interactions between the **Admin** actor and system capabilities. There are **8 main use cases** and **18 included use cases** across six domains:

1. Authentication – Login, Logout.
2. Dashboard & Monitoring – real-time stats, network graph, live bus feed.
3. Live Tracking – GPS bus positions on OLA Maps.
4. Route Management – full CRUD + CSV bulk upload.
5. Fleet Management – buses, drivers, conductors, crew assignment.
6. Analytics & Demand – revenue, tickets, route performance, demand analysis.

5.2 Class Diagram

The Class Diagram covers four logical domains with **14 Model classes**, **9 Service classes**, **4 Controller classes**, and **2 Frontend utility classes**.

Key relationships:

- **Composition:** Route $\diamond$ — Stop (stops exist only within a route).
- **Association:** Bus — Trip — Ticket.
- **Dependency:** AdminController AdminService.

5.3 Entity-Relationship (ER) Diagram

The database comprises **14 collections** in MongoDB:

Collection	Primary Key	Description
buses	busId	Fleet vehicle registry

Collection	Primary Key	Description
routes	routeId	Transit routes with embedded stops
drivers	driverId	Driver personnel records
conductors	conductorId	Conductor personnel records
trips	tripId	Trip lifecycle (busId, routeId, driverId)
tickets	ticketId	Issued travel tickets
heartbeats	_id	Real-time GPS telemetry logs
devices	deviceId	IoT ETM device registry
adminusers	_id	Admin portal accounts
appusers	userId	Passenger mobile app accounts
userbookings	bookingId	Passenger bookings with QR code
buscapacities	busId	Seat capacity per bus
offlinebatches	_id	Offline ticket sync batches
voicequeries	queryId	Multilingual voice query records

6 Object-Oriented Programming Concepts

6.1 Encapsulation

Encapsulation bundles data and behaviour together while restricting direct access to internals.

Example – Database Singleton:

Listing 1: Database class with private fields

```

1 export class Database {
2   private static instance: Database;
3   private isConnected: boolean = false; // private -- cannot be set
    externally
4
5   private constructor() { ... } // private -- no direct
    instantiation
6   public static getInstance(): Database { ... }
7   public async connect(): Promise<void> { ... }
8 }

```

Other examples:

- AdminService – private helper `getEstimatedPassengers()` not exposed to controllers.
- Mongoose Models – schema enums constrain field values (e.g., status: `active` | `inactive` | `maintenance`).
- AuthContext (frontend) – `user` and `token` state accessible only via `login()` and `logout()`.

6.2 Inheritance

BasePersonnel – shared Driver and Conductor schema:

Listing 2: IBasePersonnel interface reused by Driver and Conductor

```

1 export interface IBasePersonnel extends Document {
2   name: string; phone: string;
3   licenseNo: string;
4   status: 'active' | 'inactive' | 'on_leave';
5 }
6
7 export interface IDriver extends IBasePersonnel {
8   driverId: string; // only adds its own unique ID
9 }
10
11 export interface IConductor extends IBasePersonnel {
12   conductorId: string;
13 }

```

BaseController – abstract class inherited by all route controllers, providing `sendSuccess()` and `sendError()` helpers and enforcing `initializeRoutes()` implementation.

6.3 Polymorphism

Five authentication strategy classes implement the same `IAuthStrategy` interface but execute entirely different validation logic:

Listing 3: Polymorphic auth strategies

```

1 export interface IAuthStrategy {
2   authenticate(req, res, next): Promise<void> | void;
3 }
4
5 class AdminAuthStrategy implements IAuthStrategy { ... }
6 class DeviceAuthStrategy implements IAuthStrategy { ... }
7 class AppUserAuthStrategy implements IAuthStrategy { ... }
8 class ConductorAuthStrategy implements IAuthStrategy { ... }

```

To Express.js, all strategies look identical: `(req, res, next)`. The internal logic varies – this is runtime polymorphism.

6.4 Abstraction

The Service Layer abstracts complex MongoDB aggregation pipelines behind simple method calls:

Listing 4: Abstraction via service facade

```

1 // Controller -- simple
2 const stats = await AdminService.getRealtimeAnalytics();
3
4 // Inside AdminService -- hidden complexity
5 const heartbeats = await Heartbeat.aggregate([
6   { $match: { timestamp: { $gte: fiveMinAgo } } },
7   { $group: { _id: "$busId" } }
8 ]);
9 // ... cross-collection joins, load calculations ...

```

Summary Table:

Concept	Where Used	Example
Encapsulation	Service classes, DB Singleton, Models	<code>private isConnected</code> , schema enums
Inheritance	BasePersonnel, BaseController, Document	IDriver extends IBasePersonnel
Polymorphism	IAuthStrategy, IObservable	4 auth strategies, same interface
Abstraction	Service Layer, BaseController	AdminService hides MongoDB queries

7 Design Patterns (Gang of Four)

7.1 Singleton Pattern

Intent: Ensure a class has only one instance and provide a global access point.

Used in:

- Database class (`config/db.ts`) – one MongoDB connection pool for the whole application.
- EventBus class (`core/EventBus.ts`) – one central SSE publisher shared across all routes.
- API Client (`admin-portal/lib/api.js`) – one frontend HTTP client instance.

Listing 5: Singleton – Database class

```

1 export class Database {
2   private static instance: Database;
3   private constructor() { ... }
4
5   public static getInstance(): Database {
6     if (!Database.instance) {
7       Database.instance = new Database();
8     }
9     return Database.instance;
10  }
11 }
```

7.2 Observer Pattern

Intent: Define a one-to-many dependency so that when a subject changes state, all its observers are notified automatically.

Used in: Real-time GPS tracking via SSE.

- **Subject** – EventBus implements ISubject.
- **Observer** – SSEObserver implements IObservable, wrapping an Express Response.

Listing 6: Observer – EventBus and SSEObserver

```

1 export interface IObservable {
2   readonly id: string;
3   update(event: string, data: any): void;
4 }
5
6 export class SSEObserver implements IObservable {
7   public update(event: string, data: any): void {
```

```

8     this.response.write('data: ${JSON.stringify(data)}\n\n');
9   }
10 }
11
12 export class EventBus implements ISubject {
13   private channels: Map<string, Set<IObserver>>;
14
15   public notify(channel: string, data: any): void {
16     this.channels.get(channel)?.forEach(o => o.update(channel, data));
17   }
18 }

```

7.3 Facade Pattern

Intent: Provide a simplified interface to a complex subsystem.

Used in: The entire Service Layer. Each service class is a Facade over complex MongoDB operations. Controllers call simple methods; all aggregation pipeline logic is hidden inside.

Listing 7: Facade – AdminService hiding complexity

```

1 // Controller (simple)
2 const stats = await AdminService.getRealtimeAnalytics();
3
4 // AdminService (complex internals hidden)
5 public static async getRealtimeAnalytics() {
6   const heartbeats = await Heartbeat.aggregate([...]);
7   const crowding = { low: 0, medium: 0, high: 0 };
8   // cross-collection joins, load calculations...
9   return { active_buses, avg_speed_kmph, crowding };
10 }

```

Service Facades: AdminService, AppService, BookingService, ConductorService, IngestService, QRService, PassengerLoadService, QueryService, UploadService.

7.4 Strategy Pattern

Intent: Define a family of algorithms, encapsulate each one, and make them interchangeable.

Used in: Multi-tier authentication middleware.

Listing 8: Strategy – interchangeable auth strategies

```

1 export interface IAuthStrategy {
2   authenticate(req, res, next): Promise<void> | void;
3 }
4
5 class AdminAuthStrategy implements IAuthStrategy { ... }
6 class DeviceAuthStrategy implements IAuthStrategy { ... }
7 class AppUserAuthStrategy implements IAuthStrategy { ... }
8 class ConductorAuthStrategy implements IAuthStrategy { ... }
9
10 // Factory converts strategy to Express middleware
11 export const authenticateAdmin =
12   AuthMiddlewareFactory.create(new AdminAuthStrategy());
13 export const authenticateDevice =

```

```

14   AuthMiddlewareFactory.create(new DeviceAuthStrategy());
15
16   // Routes declare their required strategy
17   router.get('/fleet/buses', authenticateAdmin, handler);
18   router.post('/heartbeat', authenticateDevice, handler);

```

Design Patterns Summary:

Pattern	Category	Files	Purpose
Singleton	Creational	db.ts, EventBus.ts	One shared instance
Observer	Behavioural	EventBus.ts, SSEObserver.ts	SSE push notifications
Facade	Structural	services/*.service.ts	Simplified business API
Strategy	Behavioural	middleware/strategies/	Swappable auth logic

8 API Endpoints

8.1 ETM Device (/etm/v1) – X-Device-Key required

Method	Endpoint	Description
POST	/heartbeat	Submit GPS heartbeat
POST	/ticket	Issue a ticket
POST	/trip/start	Start a new trip
POST	/trip/end	End an active trip
POST	/offline-batch	Sync offline tickets

8.2 Passenger App (/app/v1) – App Token required

Method	Endpoint	Description
POST	/auth/login	Passenger login
GET	/stops	List all stops
GET	/routes/search	Search routes
GET	/routes/:id/live-buses	Live buses on route
POST	/booking	Create booking with QR
GET	/bookings	List user bookings
DELETE	/booking/:id	Cancel a booking

8.3 Conductor (/conductor/v1) – Conductor Token required

Method	Endpoint	Description
POST	/auth/login	Conductor login
GET	/trip/active	Get active trip
POST	/ticket	Issue ticket
POST	/offline-sync	Sync offline tickets

Method	Endpoint	Description
POST	/verify-qr	Verify passenger QR

8.4 Admin (/admin/v1) – Admin Token required

Method	Endpoint	Description
POST	/auth/login	Admin login
GET/POST	/routes	List / create routes
PUT/DELETE	/routes/:id	Update / delete route
POST	/routes/bulk-upload	CSV bulk import
GET	/analytics/realtime	Live operational stats
GET	/analytics/revenue	Revenue analytics
GET	/analytics/tickets	Ticket analytics
GET	/analytics/route-performance	Route performance
GET	/analytics/demand	Demand analysis
GET/POST/PUT/DELETE	/fleet/buses[:id]	Bus CRUD
GET/POST/PUT/DELETE	/fleet/drivers[:id]	Driver CRUD
GET/POST/PUT/DELETE	/fleet/conductors[:id]	Conductor CRUD
POST	/fleet/buses/:id/assign-crew	Assign driver + conductor
GET	/buses/live	Live bus positions

9 Key Features Delivered

1. **Real-Time GPS Bus Tracking** – OLA Maps integration with SSE streaming; bus markers update live.
2. **Ticketing & QR Booking** – Conductors issue tickets on-bus; passengers book via mobile app and receive QR codes verified at boarding.
3. **Offline Sync** – Conductors can issue tickets without connectivity; batches sync on reconnect.
4. **Fleet Management** – Full CRUD for buses, drivers, and conductors; crew assignment to trips.
5. **Route Management** – Stop-level route creation, CSV bulk upload, network graph visualisation.
6. **Analytics Dashboard** – Revenue charts, ticket stats, route performance, hourly demand patterns.
7. **Multi-Role Authentication** – Four independent auth strategies securing separate API segments.
8. **Voice Query Support** – Punjabi-language voice-based route queries with audio transcription.

10 Conclusion

NextStop successfully demonstrates end-to-end system design and implementation, backed by Agile SDLC discipline, UML and data-modelling diagrams, all four core OOP principles, and four proven Gang of Four design patterns. The resulting architecture is modular, maintainable, and suitable for extension into a production-grade public transit deployment.

The codebase is structured in TypeScript across 14 Mongoose models, 9 service classes, 4 controller groups, and 40+ REST API endpoints, providing a scalable foundation for future enhancements such as mobile app expansion, push notifications, and multi-city deployments.